



Aventuras do CosmoBot

Documentação Técnica e Pedagógica

Jogo educativo de matemática • 6 a 10 anos

Versão do projeto	0.2.0
Documento	Documentação técnica completa
Data	06/09/2026
Stack	React · Phaser 3 · Supabase · Vite
Finalidade	Compartilhamento com o grupo — QA e avaliadores

Sumário

Sumário	2
1. O intuito (por que o jogo existe)	3
Princípios que guiaram todo o projeto	3
2. Público e contexto de uso	3
3. Arquitetura e decisões técnicas (o “como” e o “porquê”)	3
3.1 Por que React e Phaser juntos?	3
3.2 Desenho procedural (por que quase não há imagens)	3
3.3 Fluxo de cenas do Phaser	4
3.4 Fases como dados, não como código	4
4. As 5 etapas (mecânica e modelo pedagógico)	5
Decisões pedagógicas importantes	5
4.1 Rodadas sempre aleatórias	5
5. Acessibilidade (uma prioridade, não um extra)	5
6. Áudio: dois sistemas separados	5
7. Progresso e privacidade (Supabase)	6
8. Estrutura de pastas	7
9. Como rodar e publicar	7
10. Melhorias recentes (06/09/2026)	8
10.1 Botões que “não apertavam direito” 	8
10.2 Jogo lento / travando 	8
10.3 “Caixas mal formatadas com as letras” 	8
10.4 Etapas manipuláveis (aprender fazendo) 	8
11. Limitações conhecidas e próximos passos	8
12. Roteiro de verificação (QA)	9

1. O intuito (por que o jogo existe)

O CosmoBot é um robô explorador cuja nave perdeu energia numa chuva de meteoros. Para voltar para casa, ele precisa **consertar a nave em 5 etapas**, e cada etapa é um desafio de matemática com dificuldade crescente.

O objetivo é transformar as quatro operações básicas (mais contagem) em algo **concreto e visual**: a criança não vê apenas “3 + 4”, ela vê peças dentro de caixas que se juntam, luzes que se apagam, fileiras de células e combustível sendo repartido. A matemática aparece como consequência de uma história, não como uma prova.

Princípios que guiaram todo o projeto

1. **Sem punição.** Errar nunca tira pontos nem causa “game over”. Ao errar, o jogo convida a contar de novo e destaca os itens.
2. **Sempre concreto.** Todo número tem uma representação visual clicável na tela.
3. **Acessível de verdade.** Funciona só com teclado, tem contraste alto, fonte grande e narração em voz alta.
4. **Sem dados reais da criança.** Só um apelido de brincadeira — nada de nome real, e-mail ou idade.
5. **Nunca repetitivo.** As rodadas são geradas aleatoriamente a cada partida.

2. Público e contexto de uso

- **Faixa etária:** 7 a 10 anos.
- **Onde roda:** navegador (computador e celular), inclusive dentro de um iframe do “Cruzeiro HUB”. Por isso o vite.config.js usa base: “./” (caminhos relativos, para funcionar em qualquer host/subpasta).
- **Origem:** projeto de sprint com papéis de Programação, Game Design/Pedagógico, Áudio, UX/Acessibilidade e Arte.

3. Arquitetura e decisões técnicas (o “como” e o “porquê”)

3.1 Por que React e Phaser juntos?

O jogo tem dois mundos diferentes, e cada ferramenta cuida do que faz melhor:

Camada	Ferramenta	Por quê
Telas de interface (menu, avatar, instruções, acessibilidade)	React	São telas de navegação/formulário. HTML + CSS dão acessibilidade “de graça” (foco, leitor de tela, teclado) e são fáceis de estilizar.
O jogo em si (5 fases, animações, robô)	Phaser 3	Motor de jogos 2D: cuida do canvas, animações, input de toque/mouse e escala para caber na tela.
Progresso	Supabase	Banco de dados pronto e gratuito, sem manter servidor próprio.
Build / desenvolvimento	Vite	Rápido no dev e gera um pacote otimizado para publicar.

O React controla *qual tela aparece* (App.jsx) e, ao clicar em “Jogar”, monta o Phaser dentro de um <div> pelo componente JogoCanvas.jsx. Em resumo: **React por fora, Phaser por dentro**.

3.2 Desenho procedural (por que quase não há imagens)

O robô, os cristais, as luzes, as caixas e os botões são desenhados por código em src/game/desenho.js, não carregados como imagens. Motivos:

- Não depende de artista para o jogo funcionar (a arte pode chegar depois).

- Arquivos minúsculos → carrega rápido, ideal para celular e para o iframe.
- Cores dinâmicas: o robô assume a cor que a criança escolheu.

Degradação graciosa

O `JogoCanvas.jsx` verifica por HEAD quais imagens existem em `public/assets/` (sem gerar erros 404). Se encontrar fundo, cristal, robo, planeta ou lua, usa a imagem; senão, usa o desenho procedural. O jogo nunca quebra por falta de asset.

3.3 Fluxo de cenas do Phaser

`BootScene` → `HistoriaScene` → `EstacaoScene` (HUB) → `Fase` → `EstacaoScene` → `Fase` → ... → `Final`

- **BootScene** — carrega só as imagens que realmente existem.
- **HistoriaScene** — a abertura (a historinha do CosmoBot).
- **EstacaoScene** — o “hub” (mapa da nave): entre as fases, apresenta o próximo conserto e mostra o robô avançando. No fim, a nave decola.
- **Fases** — as 5 cenas de matemática, todas herdando de `FaseBase.js`.

3.4 Fases como dados, não como código

Cada fase é um arquivo de dados (`fase01.js` ... `fase05.js`) com título, emoji, cor, enunciado, mensagem de sucesso e parâmetros de geração (mín., máx., nº de rodadas). A mecânica fica na cena; o conteúdo fica nos dados.

Por quê

Dá para ajustar dificuldade, textos e cores sem mexer na lógica do jogo — inclusive por alguém do time pedagógico.

4. As 5 etapas (mecânica e modelo pedagógico)

Princípio central

A criança FAZ a conta, não conta o resultado pronto. Só a contagem (etapa 1) se resolve olhando a tela — é a habilidade daquela idade. Nas etapas 2 a 5 a criança MANIPULA os objetos (arrasta / toca): é a ação que ensina a operação. Não dá para “passar só contando o que está na tela”.

#	Etapa (nave)	Operação	O que a criança FAZ
1	Ligar o Painel	Contagem	Conta as luzes acesas e escolhe o número
2	Abrir a Comporta	Soma	Junta as peças das duas caixas no núcleo → descobre o total
3	Encher as Baterias	Subtração	Retira (descarrega) a quantidade pedida → descobre quantas sobram
4	Ligar os Motores	Multiplicação	Enche cada motor com grupos iguais → descobre o total
5	Traçar a Rota	Divisão	Reparte o combustível igualmente entre os tanques → descobre quanto vai em cada

Decisões pedagógicas importantes

- **Manipular em vez de contar.** Somar é juntar dois grupos; subtrair é tirar; multiplicar é montar grupos iguais; dividir é repartir por igual. A resposta aparece como consequência da ação — e só então o jogo mostra o símbolo (ex.: “ $12 \div 3 = 4$ ”), ligando o que a criança fez ao que se escreve.
- **Três formas de interagir (acessível).** Toda etapa manipulável aceita **ARRASTAR**, **TOCAR** na caixa/tanque/núcleo (mais fácil no celular) e **TECLADO** (← → escolhem o alvo, Enter coloca, Backspace tira).
- **Divisão e multiplicação sempre com partes/grupos iguais.** O foco é a ideia de “repartir por igual” e “grupos iguais”, base das duas operações.
- **Quantidades pequenas de propósito.** Nas etapas de arraste os números são menores (ex.: até $3 \times 4 = 12$) para o manuseio ser agradável, sem perder o conceito.
- **Sem punição.** Repartir errado só mostra “deixe os tanques iguais” e deixa a criança ajustar — nunca há perda nem “game over”.

4.1 Rodadas sempre aleatórias

gerarRodadas.js gera as rodadas na hora, a cada início de fase, evitando repetir a mesma resposta em sequência. Assim o jogo nunca fica igual: a criança pode repetir a fase quantas vezes quiser sem decorar as respostas — exercita a operação, não a memória da tela.

5. Acessibilidade (uma prioridade, não um extra)

Recurso	Como funciona	Onde
Teclado	← → movem o foco; Enter/Espaço confirmam	FaseBase.js → configurarTeclado
Contraste alto	Paleta preta/amarela	index.css (body.contraste-alto)
Fonte grande	Aumenta a base tipográfica	index.css / JogadorContext
Narração	Lê enunciados e feedbacks em voz alta	src/lib/fala.js
Menos movimento	Desliga animações decorativas	@media (prefers-reduced-motion)

6. Áudio: dois sistemas separados

- 6. **Efeitos sonoros (src/lib/sfx.js)** — gerados na hora com a Web Audio API, sem arquivos. “Pling” de acerto, som suave de erro (nunca punitivo), fanfarra de vitória. Tudo passa por um “volume geral” que o botão Som liga/desliga.

Por quê sem arquivos: zero downloads, controle total do volume dentro do jogo, e não depende de a criança “mutar a aba”.

- 7. **Narração (src/lib/fala.js)** — voz do navegador (Web Speech API), escolhendo a melhor voz em português disponível.

Limitação: a qualidade depende do navegador/sistema. Para narração realmente natural, o ideal futuro é usar áudios gravados em public/audios/.

7. Progresso e privacidade (Supabase)

- Ao concluir cada etapa, o jogo chama salvarProgresso (src/lib/progresso.js), que grava no Supabase se as chaves estiverem configuradas (.env).
- **Sem .env, o jogo funciona igual** — apenas não grava. Isso mantém o desenvolvimento simples e não trava quem só quer jogar/testar.
- **Privacidade:** grava-se apenas o apelido fictício e o status da fase. Nenhum dado real da criança é solicitado ou armazenado.

Para ativar o Supabase

Criar projeto no Supabase, rodar supabase/tabela-progresso.sql, copiar .env.example → .env e preencher as duas variáveis.

8. Estrutura de pastas

src/	
├─ main.jsx	→ ponto de entrada do React
├─ App.jsx	→ controla qual tela aparece
├─ index.css	→ estilos + acessibilidade
├─ context/JogadorContext	→ avatar, apelido e preferências
├─ components/	→ telas em React (Menu, Avatar, Instruções...)
│ └─ JogoCanvas.jsx	→ monta o Phaser dentro do React
├─ game/	
│ └─ config.js	→ cenas + roteiro das fases + desempenho
│ └─ tema.js	→ cores e fonte
│ └─ desenho.js	→ robô, cristais, luzes, caixas e BOTÕES
│ └─ gerarRodadas.js	→ geração aleatória de cada operação
│ └─ scenes/	→ Boot, Historia, Estacao, FaseBase + 5 fases
├─ data/fases/	→ as 5 fases guardadas como dados
└─ lib/	→ supabase, progresso, sfx (efeitos) e fala (narração)

9. Como rodar e publicar

⚠ Atenção (Windows / PowerShell)

Se “npm run dev” der o erro “a execução de scripts foi desabilitada neste sistema”, é a política de segurança do PowerShell — não o projeto. Rode uma vez:

Set-ExecutionPolicy -Scope CurrentUser -ExecutionPolicy RemoteSigned

Alternativas: usar “npm.cmd run dev” ou o Prompt de Comando (cmd).

```
npm install      # só na primeira vez
npm run dev      # desenvolvimento (http://localhost:5173)
npm run build    # gera a versão final na pasta dist/
npm run preview  # testa a versão final localmente
```

10. Melhorias recentes (06/09/2026)

Correções de desempenho e usabilidade feitas após testes, validadas com o jogo rodando no navegador:

10.1 Botões que “não apertavam direito” ☒

- **Problema:** os botões de resposta às vezes não reagem ao clique/toque (o efeito de passar o mouse funcionava, mas o clique falhava), principalmente após o layout mudar (fonte carregando, redimensionar a janela, teclado do celular abrindo).
- **Causa:** a ação disparava ao soltar o toque (pointerup), que se perde quando o cache de posição do canvas fica desatualizado ou o dedo escorrega 1 px.
- **Correção (desenho.js):** a ação passou a disparar ao pressionar (pointerdown), com trava anti-duplo-toque. Resposta imediata e nenhum clique se perde.

10.2 Jogo lento / travando ☒

- **Estrelas do fundo:** eram 80 círculos, cada um com animação infinita, recriados em toda cena. Viraram 60 estrelas desenhadas num único objeto com uma só animação de brilho.
- **Desfoque de fundo (CSS):** o backdrop-filter: blur sobre um fundo animado é caro e disputava a GPU com o jogo. Foi reduzido nas telas e removido na tela do jogo.
- **Configuração do Phaser:** adicionados powerPreference: high-performance, roundPixels, limite de FPS e disableContextMenu.

10.3 “Caixas mal formatadas com as letras” ☒

- Na tela da história, o título encostava/sobreponha o texto da historinha. O painel foi reposicionado abaixo do título, com o texto centralizado dentro da caixa.
- Os enunciados das fases ganharam mais margem para não encostar nas bordas.

10.4 Etapas manipuláveis (aprender fazendo) ☒

- **Motivo:** nas versões antigas, várias etapas podiam ser resolvidas só CONTANDO o que estava na tela — a criança passava sem exercitar a operação.
- **Mudança:** as etapas de Soma, Subtração, Multiplicação e Divisão passaram a ser MANIPULÁVEIS. A criança arrasta/toca para juntar, tirar, formar grupos iguais ou repartir — e descobre o resultado ao concluir a ação. A Contagem (etapa 1) segue como introdução, pois contar é a habilidade daquela idade.
- **Faixa etária:** ampliada para 6 a 10 anos, com quantidades menores nas etapas de arraste.

11. Limitações conhecidas e próximos passos

- **Narração** depende da voz do navegador → qualidade variável. Próximo passo: áudios gravados em public/audios/.
- **Tamanho do pacote JS** (~1,6 MB) por causa do Phaser. Aceitável, mas dá para reduzir com code-splitting se necessário.
- **Arte provisória** (desenho procedural). O jogo já aceita imagens reais quando chegarem.
- **Sem painel do professor** e sem aba de geografia (fora do escopo atual).

12. Roteiro de verificação (QA)

Sugestão de checklist para a equipe de QA e os avaliadores validarem o essencial:

Item a verificar	Resultado esperado
Menu → Jogar → escolher robô e apelido	Vai para a abertura (história) sem erros
Etapa 1 (Contagem): botões de resposta	Respondem no primeiro clique/toque, sempre
Etapa 2 (Soma): juntar peças no núcleo	Ao juntar todas, revela “ $a + b = \text{total}$ ”
Etapa 3 (Subtração): descarregar células	Ao tirar a quantidade pedida, revela “ $a - b = \text{resto}$ ”
Etapa 4 (Multiplicação): encher motores	Grupos iguais → revela “ $n \times x = \text{total}$ ”
Etapa 5 (Divisão): repartir combustível	Tanques iguais → revela “ $\text{total} \div n = \text{cada}$ ”
Arrastar peças (mouse e toque)	A peça segue o dedo e cai no alvo certo
Tocar no tanque/motor/núcleo	Puxa uma peça (alternativa ao arraste)
Teclado (← → escolher, Enter, Backspace)	Coloca e tira peças sem mouse
Errar a repartição de propósito	Sem punição; mostra dica e deixa ajustar
Concluir as 5 etapas	A nave decola (celebração) e permite jogar de novo
Acessibilidade: contraste alto e fonte grande	Aplicam em todas as telas
Desempenho ao trocar de fase	Sem travar / sem engasgo perceptível
Abrir dentro do iframe do Cruzeiro HUB	Carrega e joga normalmente